

EC8203 Applied Big Data Engineering

Mini Project Assessment

Group Members:

EG/2020/3894 : Didulani P.K.S

EG/2020/3901 : Dinuk P.D

End-to-End Lambda Architecture for FinTech Fraud Detection

Table of Contents

- 1 Introduction. 3
- 2 Justification of the Selected Technology Stack 4
 - 2.1 Data Ingestion: Apache Kafka (KRaft Mode) 4
 - 2.2 Stream Processing: Apache Spark (Structured Streaming)..... 4
 - 2.3 Orchestration: Apache Airflow..... 4
 - 2.4 Storage & Sink: PostgreSQL and Apache Parquet 5
- 3 Architecture Overview 6
 - 3.1 System Diagram 6
- 4 Handling of Event Time vs. Processing Time..... 7
 - 4.1 The Challenge of Network Latency 7
 - 4.2 Implementation: Event-Time Windowing and Watermarking..... 7
 - 4.3 Watermarks for Late Data 7
- 5 Ethics Section: Privacy Implications and Data Governance 8
 - 5.1 Privacy Implications of the FinTech Scenario 8
 - 5.2 Application of Data Governance Policies 8

1 Introduction.

This report details the design and implementation of a production-grade 'FinTech Fraud Detection Pipeline' utilizing a Lambda Architecture. In modern digital wallet ecosystems, detecting fraudulent transactions with minimal latency is not just a competitive advantage but a stringent compliance requirement. The implemented system successfully detects complex fraud patterns, such as high-value anomalies and impossible travel scenarios, within seconds of occurrence, while concurrently maintaining immutable historical records for comprehensive batch analysis and automated financial reconciliation.

The pipeline simulates a high-throughput, real-world data environment where synthetic user transactions are continuously generated, streamed, processed in real-time (Speed Layer), and subsequently orchestrated for recurring batch reporting (Batch Layer).

Key features of the system

- ❖ Real-time Fraud Detection: HIGH_VALUE and IMPOSSIBLE_TRAVEL patterns detected within seconds
- ❖ Controlled Fraud Injection: 5% synthetic fraud rate for realistic testing
- ❖ Event-time Windowing: 10-minute tumbling windows with 5-minute watermarks
- ❖ Hourly ETL Pipeline: 6-hour batch reconciliation jobs
- ❖ Multi-stage Storage: Speed layer (PostgreSQL) + Batch layer (Parquet warehouse)
- ❖ Production-ready: Docker containerization with health checks

2 Justification of the Selected Technology Stack

The technology stack was carefully curated to address the rigid, dual-pronged requirements of the financial sector: zero-data-loss real-time processing and strongly consistent historical auditing.

2.1 Data Ingestion: Apache Kafka (KRaft Mode)

Financial transactions require absolute durability and ordered execution. Kafka was chosen over message queues like RabbitMQ because of its strong, distributed commit log architecture.

- Partitioning & Scalability: The `transactions` topic is split into 4 partitions, allowing parallel processing by Spark executors as data volumes grow.
- Durability & Replayability: Unlike transient queues, Kafka retains messages for 7 days. If the Spark processor fails or requires an upgrade, consumers can reliably rewind their offsets and replay the data stream from a checkpoint, ensuring zero data loss.

2.2 Stream Processing: Apache Spark (Structured Streaming)

While Apache Storm offers marginal improvements in millisecond latency (tuple-at-a-time), Spark Structured Streaming (micro-batching) was selected for its superior fault tolerance and native integration with historical data.

- Exactly-Once Semantics: In fraud detection, duplicate alerts trigger false alarm cascades and customer friction. Spark guarantees exactly-once processing semantics through write-ahead logs and offset checkpointing.
- Rich SQL Capabilities: Spark's DataFrame API provides a native, highly declarative method to define complex event-time aggregations (e.g., tumbling windows) which are notoriously difficult to implement accurately in Apache Storm without extensive boilerplate code.

2.3 Orchestration: Apache Airflow

Airflow acts as the declarative control plane for the batch layer.

- Dependency Management: The pipeline requires sequential execution:
extract raw data -> reconcile counts -> write Parquet files -> generate final CSVs.
Airflow's Directed Acyclic Graph (DAG) architecture enforces these precise topological dependencies.

- Resilience: Built-in task-level retries and XCom cross-communication ensure that transient database connection drops do not crash the entire 6-hour reconciliation workflow.

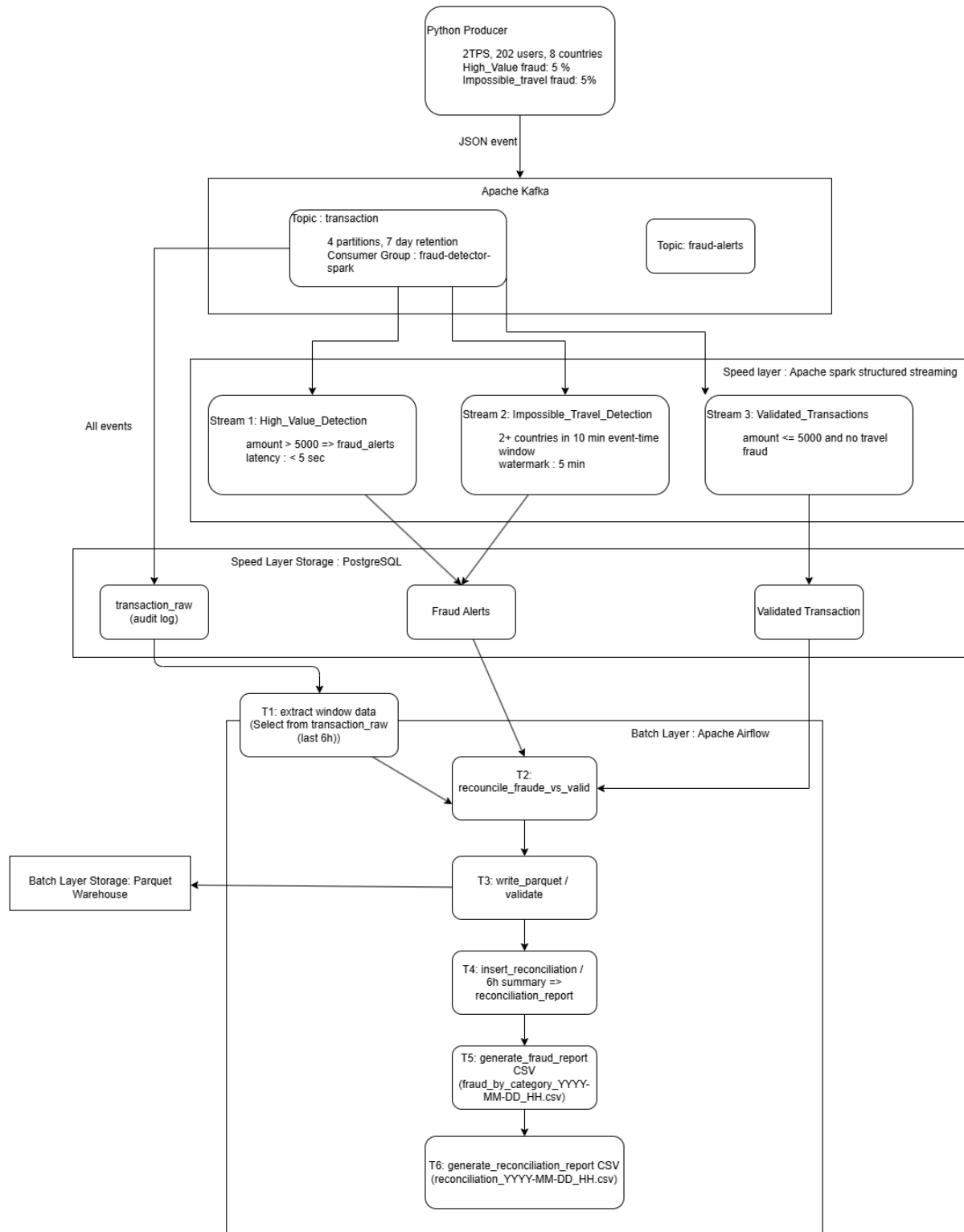
2.4 Storage & Sink: PostgreSQL and Apache Parquet

A dual-storage mechanism satisfies both Lambda layers.

- PostgreSQL (Speed Layer): Requires immediate ACID compliance and rapid indexing (``user_id``, ``event_timestamp``) to expose real-time fraud alerts to customer service dashboards via JDBC integration. NoSQL alternatives like Cassandra rely on eventual consistency, which is unsuited for immediate financial ledger updates.
- Apache Parquet (Batch/Hadoop Layer): A columnar storage format deployed over an S3/HDFS-compatible file system. Parquet drastically reduces footprint via snappy compression and enables lightning-fast analytical queries partitioned by date (``date=2026-05-09/``), perfect for historical fraud compliance auditing.

3 Architecture Overview

3.1 System Diagram



4 Handling of Event Time vs. Processing Time

In financial systems, temporal precision determines the absolute validity of an alert. A critical engineering distinction must be made between Event Time (when the transaction actually occurred on the user's mobile device) and Processing Time (when the Spark cluster receives the data via Kafka).

4.1 The Challenge of Network Latency

Transactions inherently suffer from variable network delays. If a user executes a transaction in New York at 14:00:00 (Event Time), it may be delayed in transit and only reach Kafka at 14:05:00 (Processing Time). If the system aggregates data based on Processing Time, out-of-order events will be grouped into incorrect logical windows, causing the pipeline to miss complex spatial-temporal fraud (like a second transaction appearing from London at 14:08:00).

4.2 Implementation: Event-Time Windowing and Watermarking

To resolve this, the system explicitly extracts the ISO-8601 UTC timestamp encoded by the producer within the JSON payload.

Spark defines a 10-minute Tumbling Window mapped exclusively to this extracted `event\_timestamp`.

```
windowed = (  
  df  
    .withWatermark("event_timestamp", "5 minutes")  
    .groupBy(  
      F.window("event_timestamp", "10 minutes"),  
      F.col("user_id")  
    )  
    .agg(F.collect_set("country").alias("countries_seen"))  
    .filter(F.size("countries_seen") > 1)  
)
```

4.3 Watermarks for Late Data

Because data can arrive out of order, the pipeline utilizes a 5-minute Watermark. The watermark conceptually tells the Spark engine: *"Hold the state for the current 10-minute window open until the maximum observed event time is at least 5 minutes past the end of the window."* This ensures the engine accepts delayed but legitimate messages resulting from poor 4G connectivity, incorporating them into the correct analytical window before finalizing the fraud computation.

Events arriving later than the 5-minute threshold are intentionally dropped to prevent memory leaks in the Spark state store, optimizing cluster performance.

5 Ethics Section: Privacy Implications and Data Governance

While the implemented pipeline provides strong security for user funds, continuous ingestion of location and financial data introduces severe privacy and ethical considerations. User Profiling and Surveillance in FinTech Ecosystems, presents major data governance challenges.

5.1 Privacy Implications of the FinTech Scenario

The transaction payloads continuously beam granular geospatial coordinates, precise timestamps, transaction amounts, and behavioral merchant preferences into the central cluster.

1. **Mass Surveillance & Unwarranted Tracking:** By aggregating location points ("New York, US"->"London, GB") to detect impossible travel, the system inherently creates a highly accurate, real-time map of user movements. If compromised, this data risks becoming a tool for physical stalking, state surveillance, or corporate espionage.
2. **Behavioral Profiling:** The ``merchant_category`` metric allows the enterprise to invisibly classify users (e.g., categorizing users who frequent "Healthcare" or "Liquor" merchants). This data can easily be exploited for predatory target advertising, insurance rate hikes, or unauthorized credit scoring, fundamentally violating user autonomy.
3. **Algorithmic Bias in Fraud Rules:** The hard-coded \$5,000 threshold for the ``HIGH_VALUE`` alert creates a homogenous, discriminatory standard. Lower-income individuals transacting their life savings (\$3,000) will receive no protection from the rule, while high-net-worth individuals conducting routine business may face constant false-positive account freezes. Over time, rigid geographic windowing can also flag legitimate transactions from users who frequently travel across borders for work, institutionally penalizing specific demographic cohorts.

5.2 Application of Data Governance Policies

To ethically operate this pipeline, the enterprise must implement a rigorous Data Governance Framework encompassing minimization, RBAC, and lifecycle management.

- **Data Minimization & Anonymization:** The system should aggressively obscure Personally Identifiable Information (PII) at the edge. The ``user_id`` must be salted and hashed (e.g., via SHA-256) before entering the Kafka broker, ensuring the analytics teams can group actions by user without ever knowing their real-world identity. Additionally, exact GPS coordinates must be truncated to coarse country codes immediately during Spark processing, preventing micro-level location tracking.

- Enforced Data Lifecycle (Retention Policies): Data should not exist indefinitely. The governance policy must define strict TTL (Time-To-Live) constraints:
 - Speed Layer Storage: The raw transactions table should invoke automated purge procedures, dropping event logs older than 7 days, as their utility for real-time fraud detection has expired.
 - Batch Layer Storage: Historical Parquet files must strip all location data entirely, keeping only the financial ledger components necessary for standard accounting and compliance audits (typically mandated for 7 years by financial regulators).
- Transparency and Opt-Out Architecture: Users must explicitly consent to the analysis of their location patterns for fraud detection purposes. The system architecture must accommodate a database configuration table where users can toggle location-tracking preferences, bypassing the Spark `detect_impossible_travel()` function entirely for opted-out IDs.